

Contents

1	Introduction	2
2	Image Segmentation	4
3	Unsupervised Learning	6
3.1	Latent Bernoulli Autoencoder	7
3.2	Restricted Boltzmann Machines	8
3.3	Cluster Analysis	11
4	Quantum Computing	13
4.1	Ising Model and Quadratic Unconstrained Binary Optimization	14
4.2	Adiabatic Quantum Computation and Quantum Annealing	15
4.3	DWave’s Quantum Processing Units	16
5	Methodology	20
5.1	Data Preprocessing	20
5.2	Proposed Model	21
5.2.1	Data Processing Module — LBAE	21
5.2.2	Data Clustering Module — RBM	22
6	Experiments and results	25
6.1	Dataset	25
6.2	Classical Model Training and Evaluation	26
6.3	Quantum Model Training and Evaluation	28
6.4	Segmentation Results	29
6.5	Conclusions	30
7	Summary	33
	Bibliography	34

Chapter 1

Introduction

Technological advancements in the XX century contributed to development of computer science, and the digital era has emerged. One of the modern advancements that we witness is the rapid growth of artificial intelligence, especially in the field of generative models, particularly introducing Generative Adversarial Networks in 2014 [1], and Transformer architecture in 2017 [2]. Certainly worth mentioning is the 2024 Nobel Prize in Physics for John J. Hopfield and Geoffrey E. Hinton. The motivation for the prize was as follows: “for foundational discoveries and inventions that enable machine learning with artificial neural networks”. There are more generative-based models than those two mentioned, and this work will explore the application of other models.

The Restricted Boltzmann Machines (RBM), initially introduced in 1986 under the name Harmonium [3], fall into the generative models category due to their ability to learn a probabilistic distribution over its set of inputs. RBM is a widely used machine learning technique for unsupervised [4] and supervised tasks [5]. We will use RBM as an unsupervised learning model in a pixel-level image segmentation context [6], [7].

Segmentation is a core task in computer vision. It partitions an image into regions representing different objects or materials. Segmentation makes it possible to analyze image’s structure accurately, and it has applications in many fields, such as medicine [8] or satellite image analysis [9].

In this work, we will focus on the segmentation of multispectral images. Multispectral images differ from ordinary images in that they capture information in multiple electromagnetic spectrum wavelengths [10], not just the visible range. It allows for a deeper analysis of the properties of imaged objects, as each pixel contains spectral information in different bands [11]. Research on hyperspectral imaging at the pixel-level has grown in recent years, leading to an increasing number of scientific publications in this field [11]–[15].

Quantum computing is a multidisciplinary field that includes computer science, physics, and mathematics. Quantum computing leverages the principles of quantum mechanics to solve complex problems. One possible approach to using quantum computing, alongside the gate model [16], is Adiabatic Quantum Computing (AQC) [17] and closely related quantum annealing. An interesting possibility is the use of quantum computing, both gate-based and AQC, in machine learning models [18].

D-Wave Systems Inc. is a Canadian company that has pioneered the commercial application of quantum computing. This company has developed several generations of quantum processing units (QPU) based on various architectures known as Chimera, Pegasus, and Zephyr [19].

The goal of this work is to explore the possibility of using Restricted Boltzmann Machines for pixel-level analysis of multispectral images and segmentation of those images. We also aim to use the D-Wave’s quantum annealer and compare the segmentation results with the classical RBM training method.

Chapter 2

Image Segmentation

Image segmentation is an important area of research and application in the field of computer vision. It has wide applications in many fields, such as medicine [20], remote sensing [21], and vision systems in autonomous vehicles [22]. It is the process of dividing an image into homogeneous regions. Formally, image segmentation can be defined as follows [23]. If F is the set of all pixels and $P : F \rightarrow \{\text{true}, \text{false}\}$ is a homogeneity predicate defined on groups of connected pixels, then segmentation is a partitioning of the set F into a set of connected subsets or regions $(S_1, S_2, \dots, S_n)$ such that [23]:

$$\bigcup_{i=1}^n S_i = F \quad \text{with} \quad S_i \cap S_j = \emptyset, i \neq j. \quad (2.1)$$

The homogeneity predicate $P(S_i) = \text{true}$ for all regions S_i and $P(S_i \cap S_j) = \text{false}$, when S_i is adjacent to S_j . Set F is a digital image defined as [23]:

$$F_{W \times H} = [f(x, y)]_{W \times H}, \quad (2.2)$$

where W and H are image dimensions and $f(x, y) \in G_L = \{0, 1, \dots, L - 1\}$ is a set of discrete levels of the feature value, (x, y) denotes the spatial coordinate. Equation (2.2) refers to monochrome images, although it could be extended to commonly used images in RGB or other color spaces by adding channel or band dimension. The extended form of Equation (2.2) can be defined as:

$$F_{W \times H \times B} = [f(x, y, \lambda)]_{W \times H \times B}, \quad (2.3)$$

where B is a band dimension. The Equation (2.3) represents the hyperspectral or multispectral image on which we intend to perform segmentation.

The task of image segmentation can be approached with a variety of algorithms,

including both classical methods [24] and those based on deep learning [25]. Among classical algorithms that are often used for segmentation are approaches like [24]:

- thresholding,
- region-based,
- edge-detection,
- clustering.

Clustering is an unsupervised machine learning class of algorithms that allows models to learn patterns in data without explicit labels. Segmentation with the classical clustering algorithm can be implemented using *e.g.*, k -means [26] or Agglomerative Hierarchical Clustering [27]. Both of these algorithms will be discussed in the following chapters, as they will be used to set a baseline for our model to exceed and to recognize the higher hierarchy of clusters returned by our model.

The following chapters will also discuss clustering using deep learning models, as the neural network architecture presented in this dissertation derives from deep learning.

Chapter 3

Unsupervised Learning

Artificial intelligence is a technology at the intersection of mathematics and computer science. It includes a wide spectrum of methods and algorithms that enable machines to learn, resulting in a wide range of applications [28], [29]. The origins of artificial intelligence are considered to be the Perceptron model presented conceptually in [30] and developed in [31]. Literature [28] proposes the following division of artificial intelligence algorithms:

- supervised learning,
- unsupervised learning,
- reinforcement learning.

Unsupervised learning is a fundamental approach in machine learning that allows models to learn patterns in data without explicit labels. The goal of learning in this context is to uncover the underlying structure of the dataset. The classic problems in which unsupervised learning is applied are clustering and dimensionality reduction. Clustering can be performed using a variety of algorithms [32], including partitioning methods, density-based and hierarchical methods. Dimensionality reduction has traditionally been achieved using principal component analysis (PCA) [29], but autoencoders have also proven to be highly effective for this task [33].

In the context of this work, we will use several unsupervised learning algorithms, one to enable sufficient representation learning and dimensionality reduction and other clustering algorithms to perform image segmentation.

3.1 Latent Bernoulli Autoencoder

Autoencoders are artificial neural networks used for unsupervised representation learning. Representation learning or feature learning is a process during which algorithms extract meaningful patterns from data to create representations usually in a lower dimension than the original data.

The autoencoder architecture is built from two separate networks. The first network, referred to as the encoder, executes the encoding operation $h = f(x)$, while the second one, known as the decoder, carries out the decoding operation $r = g(h)$. An autoencoder whose latent space h dimension is lower than the input data dimension is called undercomplete [29]. Learning an undercomplete representation forces the autoencoder to capture the most significant features of the training data. When the encoder and decoder are linear and the loss function is the mean squared error, an undercomplete autoencoder learns to capture the same subspace as PCA. In this scenario, an autoencoder trained to reconstruct its input ends up learning the principal subspace of the training data as a byproduct. Autoencoders with nonlinear encoder transformation f and nonlinear decoder transformation g can therefore learn a more flexible and powerful nonlinear extension of PCA [29].

Autoencoder networks may be viewed as special cases of feed-forward networks and can be trained similarly, especially using gradient descent and back-propagation techniques. The learning process in an autoencoder involves minimizing a loss function, denoted by L , which quantifies the difference between the input and the reconstructed output [29]. A loss function is generally defined as:

$$L(x, g(f(x))), \quad (3.1)$$

where L is a loss function penalizing $g(f(x))$ for being dissimilar from x . The loss function is typically defined as the mean squared error (MSE):

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (\mathbf{x}_i - \hat{\mathbf{x}}_i)^2, \quad (3.2)$$

where $\mathbf{x}_i$ is an input data — a vector of features and $\hat{\mathbf{x}}_i$ is the $\mathbf{x}_i$ reconstruction, which is obtained as an autoencoder output [29].

Latent Bernoulli Autoencoder (LBAE) belongs to the generative models' category, and its idea is based on a class of autoencoders called Variational Autoencoders (VAE) [9]. The interesting LBAE property is that the hidden layer is binarized by [9]:

$$h_i = \begin{cases} 1, & \text{if } \tanh(\mathbf{z}_i) \geq 0 \\ -1, & \text{if } \tanh(\mathbf{z}_i) < 0 \end{cases}, \quad (3.3)$$

where $\mathbf{z}_i$ is the input data transformed by the encoder. The importance of this property will be highlighted in the methodology section of this work.

3.2 Restricted Boltzmann Machines

In contrast to traditional approaches in machine learning and deep learning, structured probabilistic models offer a different paradigm. These models focus on representing probability distributions through graph-based structures, defined as:

$$G = (V, E), \quad (3.4)$$

where V represents vertices and E represents edges in a given graph G . Each vertex represents a random variable, and each edge represents direct interaction between random variables [29]. The type of connection between vertices defines whether the model is directed or undirected.

Restricted Boltzmann Machines are undirected graphical, energy-based models that contain a layer of observable variables and a layer of latent variables. Referring to the layer of observable variables, we will speak of the visible layer marked as $\mathbf{v}$, and referring to the layer of latent variables, we will speak of the hidden layer marked as $\mathbf{h}$ [29][34].

Based on [5], [29], we can define the joint probability distribution of the RBM as follows:

$$P(\mathbf{x}) = \frac{1}{Z} e^{-E(\mathbf{x})}, \quad (3.5)$$

where $\mathbf{x} \in \{0, 1\}^n$. Equation (3.5) can be also written with respect to $\mathbf{v}$ and $\mathbf{h}$ layers as [29]:

$$P(\mathbf{v}, \mathbf{h}) = \frac{1}{Z} e^{-E(\mathbf{v}, \mathbf{h})}, \quad (3.6)$$

where $\mathbf{v} \in \{0, 1\}^n$ and $\mathbf{h} \in \{0, 1\}^n$. The energy function for the RBM is given by:

$$E(\mathbf{v}, \mathbf{h}) = -\mathbf{b}^T \mathbf{v} - \mathbf{c}^T \mathbf{h} - \mathbf{v}^T \mathbf{W} \mathbf{h}, \quad (3.7)$$

where $\mathbf{b}, \mathbf{c} \in \mathbb{R}^n$ are biases and $\mathbf{W} \in \mathbb{R}^{n \times n}$ is a connection matrix. The following equation gives the partition function:

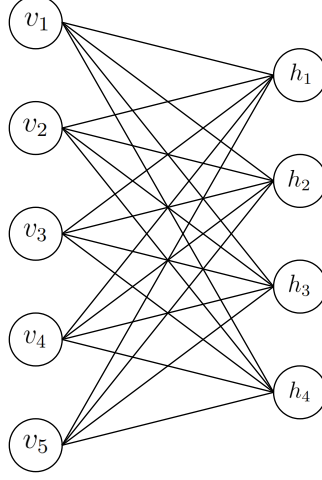


Figure 3.1: The general architecture of the RBM graph. Here, $v_i \in \{1, 2, \dots, 5\}$ denote vertices — neurons in visible layer $\mathbf{v}$ and $h_i \in \{1, 2, 3, 4\}$ denote neurons in hidden layer $\mathbf{h}$. Each vertex connects through edges to all vertices in the other set but to none in its own set.

$$Z = \sum_{\mathbf{v}, \mathbf{h}} e^{-E(\mathbf{v}, \mathbf{h})}. \quad (3.8)$$

The partition function is intractable to compute and the joint probability distribution being a function of Z is also computationally intractable. However, due to RBM's graph structure, the conditional distributions $P(\mathbf{h}|\mathbf{v})$ and $P(\mathbf{v}|\mathbf{h})$ are simple to compute. $P(\mathbf{h}|\mathbf{v})$ is given by:

$$P(\mathbf{h} | \mathbf{v}) = \frac{P(\mathbf{v}, \mathbf{h})}{P(\mathbf{v})}, \quad (3.9)$$

where $P(\mathbf{v})$ is given by:

$$P(\mathbf{v}) = \frac{\sum_{\mathbf{h}} e^{-E(\mathbf{v}, \mathbf{h})}}{Z}. \quad (3.10)$$

By substituting values from equations (3.6), (??), (3.10) into (3.9) we receive:

$$P(\mathbf{h} | \mathbf{v}) = \frac{e^{\sum_j c_j h_j + \sum_j (\mathbf{v}^T \mathbf{W})_j h_j}}{Z'}, \quad (3.11)$$

where Z' is:

$$Z' = \sum_{\mathbf{h}} e^{\mathbf{c}^T \mathbf{h} + \mathbf{h}^T \mathbf{W} \mathbf{v}}, \quad (3.12)$$

and therefore,

$$P(\mathbf{h} \mid \mathbf{v}) = \frac{1}{Z'} \prod_j e^{\sum_j c_j h_j + (\mathbf{v}^T \mathbf{W})_j h_j}. \quad (3.13)$$

We can denote this as:

$$\tilde{P}(\mathbf{h} \mid \mathbf{v}) = e^{c_j h_j + (\mathbf{v}^T \mathbf{W})_j h_j}. \quad (3.14)$$

It is now a simple matter of normalizing the distributions over the individual binary h_j :

$$P(h_j = 1 \mid \mathbf{v}) = \frac{\tilde{P}(h_j = 1 \mid \mathbf{v})}{\tilde{P}(h_j = 0 \mid \mathbf{v}) + \tilde{P}(h_j = 1 \mid \mathbf{v})}, \quad (3.15)$$

$$P(h_j = 1 \mid \mathbf{v}) = \frac{e^{c_j + (\mathbf{v}^T \mathbf{W})_j}}{e^0 + e^{c_j + (\mathbf{v}^T \mathbf{W})_j}}, \quad (3.16)$$

$$P(h_j = 1 \mid \mathbf{v}) = \frac{1}{1 + e^{-c_j - (\mathbf{v}^T \mathbf{W})_j}}. \quad (3.17)$$

Using the formula for the sigmoid function:

$$\sigma(x) = \frac{1}{1 + e^{-x}}, \quad (3.18)$$

we can denote Equation (3.17) as:

$$P(h_j = 1 \mid \mathbf{v}) = \sigma(c_j + (\mathbf{v}^T \mathbf{W})_j). \quad (3.19)$$

Similarly we can obtain $P(v_j = 1 \mid \mathbf{h})$ by:

$$P(v_i = 1 \mid \mathbf{h}) = \sigma(b_i + (\mathbf{h}^T \mathbf{W})_i). \quad (3.20)$$

RBM training is based on maximizing the log-likelihood of the training data. The log-likelihood is given by [5]:

$$l(\theta) = \sum_{t=1}^N \log P(\mathbf{v}^{(t)}), \quad (3.21)$$

and this equation can be denoted as [5]:

$$l(\theta) = \sum_{t=1}^N \log \sum_{\mathbf{h}} e^{-E(\mathbf{v}^{(t)}, \mathbf{h})} - N \cdot \log \sum_{\mathbf{v}, \mathbf{h}} e^{-E(\mathbf{v}, \mathbf{h})}, \quad (3.22)$$

where $\theta = \{\mathbf{W}, \mathbf{b}, \mathbf{c}\}$ and $\mathbf{v}^{(t)}$ is a sample from the training dataset with N records. The gradient of the log-likelihood is given by [5]:

$$\nabla_{\theta} l(\theta) = \sum_{t=1}^N \frac{\sum_{\mathbf{h}} e^{-E(\mathbf{v}^{(t)}, \mathbf{h})} \nabla_{\theta} (-E(\mathbf{v}^{(t)}, \mathbf{h}))}{\sum_{\mathbf{v}, \mathbf{h}} e^{-E(\mathbf{v}, \mathbf{h})}} - N \cdot \frac{\sum_{\mathbf{v}, \mathbf{h}} e^{-E(\mathbf{v}, \mathbf{h})} \nabla_{\theta} (-E(\mathbf{v}, \mathbf{h}))}{\sum_{\mathbf{v}, \mathbf{h}} e^{-E(\mathbf{v}, \mathbf{h})}}, \quad (3.23)$$

$$\nabla_{\theta} l(\theta) = \sum_{t=1}^N \langle \nabla_{\theta} (-E(\mathbf{v}^{(t)}, \mathbf{h})) \rangle_{P(\mathbf{h}|\mathbf{v}^{(t)})} - N \cdot \langle \nabla_{\theta} (-E(\mathbf{v}, \mathbf{h})) \rangle_{P(\mathbf{v}, \mathbf{h})}, \quad (3.24)$$

where $\langle \cdot \rangle_{P(\mathbf{v}, \mathbf{h})}$ is the expectation value with respect to the distribution $P(\mathbf{v}, \mathbf{h})$. This is a decomposition into positive phase and negative phase. The RBM is an example of a model for which positive-phase and negative-phase are, respectively, easy and difficult [5], [29].

Negative-phase $\nabla_{\theta}(-E(\mathbf{v}, \mathbf{h}))$ is hard to compute but its approximation can be obtained using the Monte Carlo Markov Chain methods (MCMC) [29]. One of the MCMC algorithms family is the contrastive divergence (CD) algorithm. CD uses the dependencies described in the Equations (3.19) and (3.20). Alternatively, the negative-phase can be calculated using samples drawn from a quantum annealer [5]. The use of annealing can lead to more efficient sampling than MCMC algorithms, which can struggle to correctly approximate the negative phase of the gradient [29]. While CD relies on limited Gibbs sampling steps and can get stuck in local minima, annealing explores more global configurations, improving sampling accuracy and learning dynamics in challenging scenarios.

3.3 Cluster Analysis

Cluster analysis is a category of unsupervised learning algorithms that seek to divide a given set of objects into homogeneous clusters [35]. In the context of image segmentation, cluster analysis divides an image into regions with similar properties.

Clustering algorithms are divided into [32]:

- partitional clustering,
- hierarchical clustering,

- density-based clustering.

Each of these approaches offers unique advantages in dealing with different underlying data structure. Partitional clustering methods, attempt to directly partition a dataset into k distinct clusters. Hierarchical clustering methods construct a hierarchy of clusters by successively merging smaller clusters (agglomerative approach) or splitting larger clusters (divisive approach). Density-based methods define clusters based on high-density regions, effectively identifying clusters of arbitrary shapes and distinguishing noise or outliers [32].

The following chapters of this work will be devoted to an algorithm based on partial clustering, which is k -means, and a hierarchical clustering algorithm, which is Agglomerative Hierarchical Clustering (AHC).

Chapter 4

Quantum Computing

Quantum computing is a domain that has gained immense interest in recent years in both the scientific and industrial sectors [36]–[39]. Taking advantage of the unique properties of quantum mechanics, such as superposition and entanglement, quantum computers promise a breakthrough in information processing, offering computational capabilities unavailable to classical computers.

Traditional computers operate on bits, which can take the value of 0 or 1. In contrast, quantum computers operate on qubits (quantum bits), which, due to the phenomenon of superposition can exist in a state that is a linear combination of states “0” and “1”, enabling quantum computers to process information in different ways compared to classical computers. In addition, quantum entanglement allows the states of qubits to be non-locally connected, leading to performing some computational tasks exponentially faster than classical algorithms [40].

A quantum system evolves according to the Schrödinger equation [18]:

$$i\hbar \frac{d}{dt} |\psi(t)\rangle = H(t) |\psi(t)\rangle, \quad (4.1)$$

where $|\psi\rangle$ is the quantum state of the system at the time t and $H(t)$ is the time-dependent Hamiltonian of a given system. $\hbar$ is Planck's constant. Hamiltonian operator corresponds to the total energy of the system. The possible energies the system can have are the eigenvalues of the H operator [18], [41].

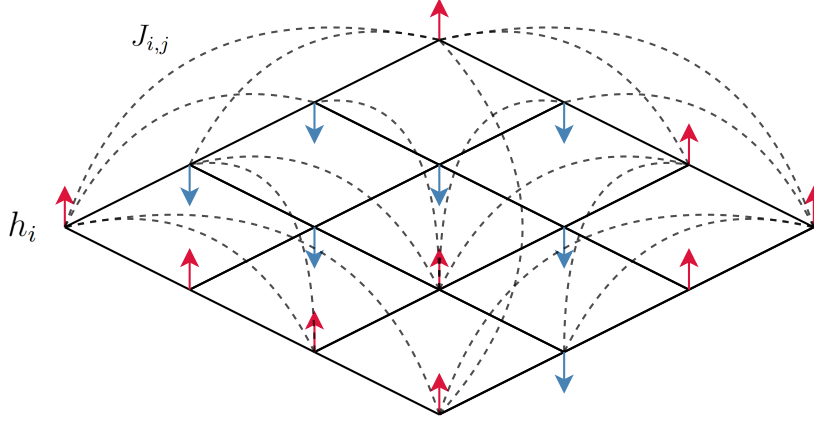


Figure 4.1: Symbolic representation of Ising model defined on the graph with $N = 16$ vertices. $J_{i,j}$ denotes coupling strength associated with an edge between i -th and j -th node, and h_i denotes qubit bias associated with i -th node.

4.1 Ising Model and Quadratic Unconstrained Binary Optimization

Ising model classically refers to disordered magnetic alloys containing strongly interacting ions immersed in a weakly interacting substrate [7]. We can interpret such an arrangement as randomly interacting spins up ($\uparrow$) or down ($\downarrow$) placed on lattice nodes. We can denote such an arrangement by an undirected graph $G = (V, E)$ [42]. For each vertex, we assign a spin variable s_i and a local magnetic field h_i . To each edge, we assign an interaction strength J_{ij} . For such a system we can define a Hamiltonian:

$$H_{\text{ising}}(s) = - \sum_{\langle i,j \rangle} J_{ij} s_i s_j - \sum_i h_i s_i, \quad (4.2)$$

where $s_i \in \{-1, 1\}$, $J_{ij} \in \mathbb{R}$, $h_i \in \mathbb{R}$. An Ising model with fixed coefficients is used to find the given system's ground state, its lower energy state of a system's Hamiltonian. However, the Ising model is not the only tool to find the given system's ground state; the other one is Quadratic Unconstrained Binary Optimization (QUBO).

What distinguishes QUBO from the Ising model is that QUBO uses binary variables $q_i \in \mathbb{B} = \{0, 1\}$, instead of $s_i \in \{-1, 1\}$ spin variables. QUBO problem can be defined as minimizing the objective function:

$$\min_{q \in \{0,1\}^n} f_Q(q). \quad (4.3)$$

The objective function $f_Q : \mathbb{B}^n \rightarrow \mathbb{R}$ is in the form given by:

$$f_Q(x) = q^T Q q, \quad (4.4)$$

where $Q \in \mathbb{R}^{n \times n}$ is upper triangular matrix, whose entries Q_{ij} define a weight for each pair of indices $i, j \in \{1, 2, \dots, n\}$. The objective function can be transformed to the following form:

$$f_Q(q) = \sum_i Q_{ii} q_i + \sum_{i < j} Q_{ij} q_i q_j, \quad (4.5)$$

which makes the QUBO problem and the Ising model essentially equivalent. To express the Ising model as a QUBO problem, we introduce the substitution $s_i = 2q_i - 1$ and we assign a_i and b_{ij} symbols to denote respectively linear and quadratic QUBO coefficients. Values for a_i and b_{ij} are as follows [42]:

$$a_i = 4J_{ij}, \quad b_{ij} = 2h_i - 2 \sum_{\langle i, j \rangle} J_{ij}, \quad (4.6)$$

what results in the following form of f_Q :

$$f_Q(q) = \sum_i a_i q_i + \sum_{i < j} b_{ij} q_i q_j, \quad (4.7)$$

which differs from (4.2) by a constant value, although it is irrelevant to the optimization process [42].

4.2 Adiabatic Quantum Computation and Quantum Annealing

The core concept of Adiabatic Quantum Computing (AQC) relies on the adiabatic theorem [43], which states that if a quantum system begins in the ground state of its initial Hamiltonian and the Hamiltonian changes slowly enough, the system will remain in its ground state throughout the evolution.

The time-dependent Hamiltonian $H(t)$ is constructed from H_{init} and H_{target} . The adiabatic evolution of a state transforms the known ground state of the H_{init} into the unknown ground state of the H_{target} . $H(s)$ is given by [44], [45]:

$$H(s) = (1 - s)H_{\text{init}} + sH_{\text{target}}, \quad (4.8)$$

where $s = \frac{t}{T} \in [0, 1]$ and T is the total evolution time, and it controls the rate at which $H(s)$ or $H(t)$ is varying. In AQC, the slower the evolution process, the better the approximate solution is obtained [46]; this is why, in practice, the annealing approach is being used.

Annealing is a process in metallurgy that involves slowly cooling metal to achieve a state with the lowest possible internal energy, resulting in greater strength and structural stability [47]. In this process, the metal atoms move freely at high temperatures, and the slow reduction of temperature allows them to gradually arrange themselves in a stable configuration.

Simulated annealing (SA) is an optimization algorithm inspired by this physical process. SA treats the optimization problem as a physical system, with energy representing the objective function value and temperature controlling the probability of accepting inferior solutions. The algorithm starts at a high temperature, where energy-increasing movements are allowed, and then the temperature slowly decreases to find the global minimum of the objective function [48].

Quantum annealing (QA) is an extension of the idea of simulated annealing that uses quantum effects to search the solution space. Unlike SA, which relies on classical temperature perturbations, QA uses quantum fluctuations to overcome energy barriers. Optimization problems, solvable by QA, are most often represented in terms of an Ising model. Although Quantum Annealing, in principle, follows similar scheme as Adiabatic Quantum Computing, the difference is that system evolution in QA is not necessarily adiabatic [49].

Quantum annealing has the potential to be used for data clustering by transforming the optimization problem of the clustering cost function into the form of a quadratic binary optimization problem. QA makes it possible to search the solution space more efficiently and deal better with local minima compared to classical methods such as k-means [44].

4.3 DWave’s Quantum Processing Units

D-Wave uses various architectures in its processors. The basic topology is Chimera and its architecture is designed to provide flexibility in mapping optimization problems onto a physical quantum system [18]. However, the graph structure, like that of Chimera, introduces some limitations, *e.g.*, not all qubits are directly connected to each other.

Qubits in the Chimera topology are placed on a square grid of unit cells. Each cell includes two groups of four qubits and each qubit is connected to four qubits from the opposite group forming $K_{4,4}$ complete bipartite graph, illustrated in the Figure ??.

Such cells are then combined into a two-dimensional grid, allowing the system to scale to a larger number of qubits and enable more complex interactions. Chimera layout is denoted by C_n , where n is the width and the height of the grid and in such configuration, each qubit is connected to a maximum of 6 qubits, the total number of qubits is $8n^2$ and the total number of couplers is $16n^2 + 8(n-1)^n$ [42]. The couplers connecting qubits in the same unit cell are called internal and the couplers connecting qubits belonging to different cells are called external. An example of Chimera topology is shown in the Figure ??.

The Pegasus P_3 architecture, illustrated in Figure 4.2, represents the next generation of the Chimera topology. This topology introduces some upgrades to qubit connectivity. Firstly, the internal couplers not only connect qubits within the same unit cell but also link qubits in adjacent unit cells. Secondly, a new type of connection, referred to as odd couplers, is introduced within the unit cells. Each Pegasus unit cell consists of 24 qubits organized into three Chimera unit cells. The Pegasus topology, denoted by P_n , features n rows and n columns of unit cells and contains a total of $24n(n-1)$ qubits [42].

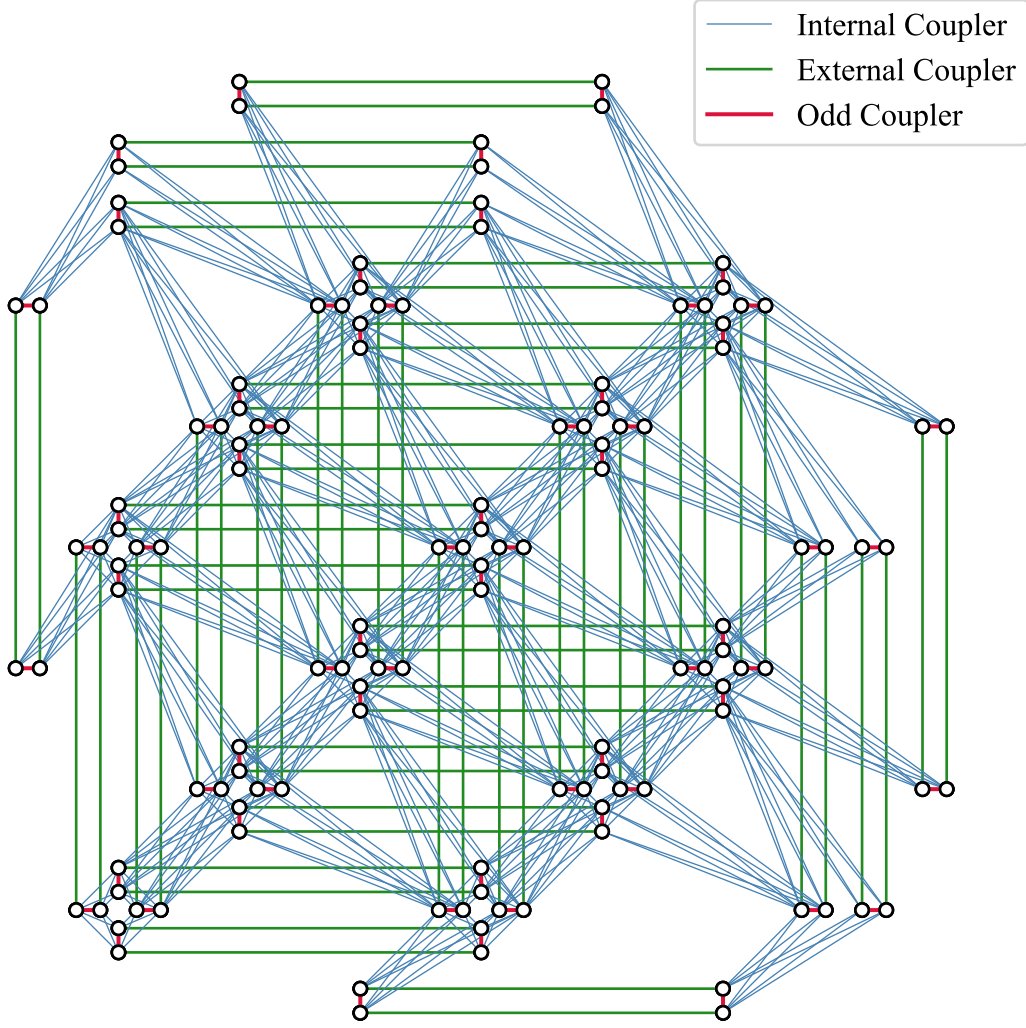


Figure 4.2: Layout of the Pegasus P_3 topology. The internal, external, and odd couplers are marked with blue, green, and red, respectively.

DWave's QPU uses Ising Hamiltonian from Equation (4.2) in a transformed form [19]:

$$H_{\text{ising}}(s) = \frac{-A(s)}{2} \left(\sum_i \hat{\sigma}_x^{(i)} \right) + \frac{B(s)}{2} \left(\sum_i h_i \hat{\sigma}_z^{(i)} + \sum_{i>j} J_{i,j} \hat{\sigma}_z^{(i)} \hat{\sigma}_z^{(j)} \right), \quad (4.9)$$

where $\hat{\sigma}_x^{(i)}$, $\hat{\sigma}_z^{(i)}$, $\hat{\sigma}_z^{(j)}$ are Pauli matrices on qubit q_i , h_i and $J_{i,j}$ are respectively the qubit biases and coupling strengths, $s = \frac{t}{T}$, where t is time, and T is the total time of the anneal. The relative energy of each state depends on the biases of qubits and couplers between them. The $A(s)$ is the tunneling energy, and the $B(s)$ is the problem Hamiltonian energy at s . Both are expressed as energies in units of Joules, as is standard for a Hamiltonian. A

linear anneal sets $s = \frac{t}{T}$, where t is time, and T is the total time of the anneal. At $t = 0$, $A(0) > B(0)$, which leads to the quantum ground state of the system where each spin is in a delocalized combination of its classical states. As the system is annealed, A decreases and B increases until T when the final state of the qubits represents a low-energy solution to a problem Hamiltonian.

Hamiltonian from Equation (4.9) is composed of two parts: H_{init} and H_{final} :

$$H_{\text{init}} = \frac{-A(s)}{2} \left(\sum_i \hat{\sigma}_x^{(i)} \right), \quad (4.10)$$

$$H_{\text{final}} = \frac{B(s)}{2} \left(\sum_i h_i \hat{\sigma}_z^{(i)} + \sum_{i>j} J_{i,j} \hat{\sigma}_z^{(i)} \hat{\sigma}_z^{(j)} \right). \quad (4.11)$$

$A(s)$ represents the tunneling energy, and $B(s)$ refers to the Hamiltonian energy of the problem in a given time step s .

Chapter 5

Methodology

This chapter will present a detailed concept of a neural network model designed for the segmentation of multispectral images. In what follows, we discuss image data preprocessing, implementation details of the model’s components, their integration, and methods of their evaluation, especially in the context of multispectral image segmentation.

5.1 Data Preprocessing

The preparation of spectral data for processing by the neural network model proposed later in this work included transformations that are standard in the field of machine learning.

The documentation of the dataset [50] describes some of the hyperspectral channels as noisy, and we decided that the indicated spectral bands should be removed from further processing. We also decided to remove from hyperspectral images pixels whose position (x, y) corresponded to the label “0” (which is the background) in the ground truth image; by such action, we wanted to achieve better clustering results because the background in the studied dataset is not homogeneous — the neural network model would try to assign background pixels to different classes.

The following data transformations were to perform normalization, shuffle the data, and divide the data into training, validation, and test sets. Scaling to the interval $\langle 0, 1 \rangle$ was chosen as a standard normalization method in machine learning [28] but was also used for processing hyperspectral images [45]. Data shuffling was used to reduce the influence of sequential pixel order. Data partitioning was done according to the Pareto principle: first, the data were divided into a training set and a test set in a ratio of 0.8 : 0.2, and then the training set was further split into a training and validation set, also in a ratio of 0.8 : 0.2.

5.2 Proposed Model

Our proposed model, shown in Figure 5.1, accepts preprocessed input data as described above section of this work. Our proposed model consists of two main parts. The part, that this work aim to study is the Restricted Boltzmann Machines (RBM) part, and the second one is the Latent Bernoulli Autoencoder (LBAE) part. RBM requires binary data as input, so the data must be encoded into binary form. In our model, LBAE’s encoder will be used for this task as it is proven to work well in similar task [51]. The data flow starts with LBAE, so we will start describing our proposed model with it.

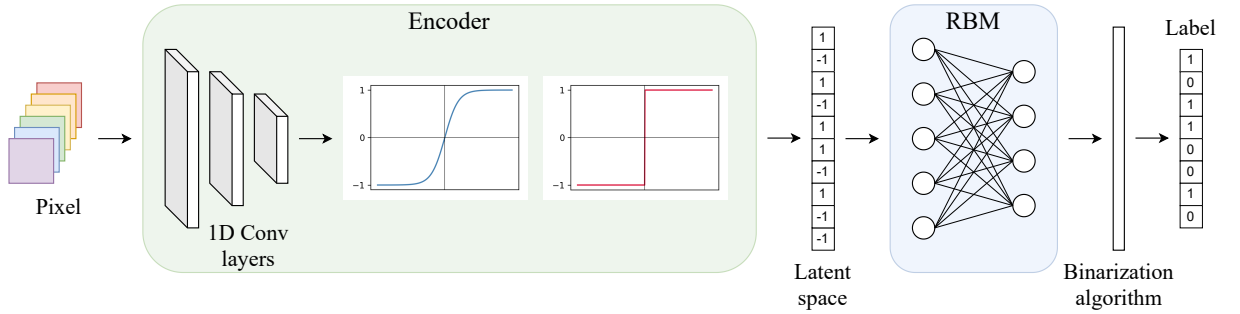


Figure 5.1: Proposed model pipeline. The pipeline processes spectral pixel data through the LBAE encoder, which consists of one-dimensional convolutional layers followed by a $\tanh$ activation and binarization (as described in Equation (3.3)). The resulting binary representation is forwarded into the visible layer of the RBM, where the neuron’s activation probabilities in a hidden layer are binarized, and the resulting binary vector is considered as a label.

5.2.1 Data Processing Module — LBAE

LBAE is an autoencoder which was introduced in [9] and implemented in [51]. It contains convolutional layers in both encoding and decoding parts. Our approach relies on the work done in the [51]. However, our approach explores the possibility of pixel-level analysis in the context of hyperspectral images, so we had to adjust [51] implementation of the LBAE. This adjustment consisted of changing the convolution layers from two to one-dimensional. The rest of the network and its layers will stay the same as in [51] as this project obtained satisfactory performance. LBAE will be trained using techniques such as back-propagation and gradient descent as described in Section 3.1. Our LBAE transforms the spectral (already preprocessed) data into a binary representation. Input data is a 112-element vector, which will be transformed through convolutional layers. Due to the undercomplete architecture of this autoencoder, the dimension of this vector shrinks accordingly to each convolution layer as it is described by a given formula [52]:

$$L_{\text{out}} = \left\lfloor \frac{L_{\text{in}} + 2 \times \text{padding} - \text{dilation} \times (\text{kernel size} - 1) - 1}{\text{stride}} + 1 \right\rfloor, \quad (5.1)$$

where L_{out} is the output length of the vector applying the convolutional layer, and L_{in} is the input vector length. The term padding refers to the number of elements added to each input side to control the output length. The dilation factor specifies the spacing between elements in the convolution kernel, effectively increasing the kernel’s receptive field, kernel_size represents the size of the convolutional kernel, or the number of samples covered by the kernel in one pass, while stride denotes the number of steps the kernel moves with each application.

Table 5.1: Parameters of the convolutional layers in the LBAE encoder.

Layer	Padding	Dilation	Kernel Size	Stride
Conv1	1	1	3	1
Conv2	1	1	4	2
Conv3	1	1	4	2
Conv4	1	1	3	1

The vector that we aim to obtain, which is a vector in the latent space of our autoencoder, has 28 binary elements. We postulate that if the values of both metrics, which we will observe, the Euclidean distance and the spectral angle distance computed between the input spectral vector and reconstructed spectral vector, are low, the hidden representation in the latent space is good. This binary representation will serve as an input to the RBM.

5.2.2 Data Clustering Module — RBM

RBM is a neural network that accepts binary data as its input, and by the LBAE encoding part, we can obtain a binary representation of the original data. The number of neurons in the visible layer of our RBM is determined by input data dimension and LBAE encoder convolutional layers; in the case of our experiments, that would be 28 neurons.

We will conduct experiments to train the RBM model with a fixed number of neurons in its visible layer and a changing number of neurons in its hidden layer $h_i \in \{3, 4, \dots, 28\}$ using CD-1 algorithm. Our approach is to take RBM’s hidden layer neurons activation probabilities calculated with Equation (3.19) and binarize those probabilities, and then the obtained binary vector is a label. The binarization threshold will be selected as follows: for each $th_i \in \{0.1, 0.2, \dots, 0.9\}$, we will compute adjusted rand score (ARS) between true labels and predicted labels. Then we will check for which threshold we

obtained the highest ARS, and for this threshold, we compute other metrics. Then, while having trained RBM models, we proceed to determine the best one by comparing them via V-measure, a meta metric whose value depends on homogeneity, completeness, and an additional β parameter. Authors of [53] do not suggest how to choose β value, however it is said by them that $\beta \in [0, 1]$ promotes homogeneity. We will conduct an experiment that explores, for which model we obtained, the highest value of V-measure for $\beta_i \in \{0, 0.01, \dots, 1\}$. The RBM architecture that most frequently obtains the highest V-measure scores will be selected as the best one, and we will proceed to experiments with simulated annealing (SA) and quantum annealing (QA) on this architecture.

The utilization of annealing to train Restricted Boltzmann Machines represents a novel approach to optimizing model parameters by exploiting quantum effects [5], [40]. RBM training on the annealer starts with determining the probability distribution of the hidden RBM layer for a given set of visible data. The RBM coefficients, such as the weights and the biases of the visible and hidden units, are then transformed into a Quadratic Unconstrained Binary Optimization (QUBO) problem. The QUBO is defined by a quadratic function, where the linear elements correspond to the biases, and the quadratic elements correspond to the weights between the neurons. The generated QUBO is then sampled by an annealer, which finds the variables' values corresponding to the objective function's minimization. In this context, the annealer acts as a probabilistic sampler, providing samples of the values of hidden and visible RBM units based on energy minimization. The values of these samples are then used to update the RBM weights and biases. The algorithm gradually adjusts the model by comparing the distributions sampled by the annealer with those inferred from the data, allowing the hidden structures to be correctly represented in the data. Using the annealer in this context can help more efficient sampling in cases where gradient methods become insufficient or when the problem has a complex energy space, making sampling more difficult.

Quantum annealer that we were granted access to uses Hamiltonian in the form presented in Equation (4.9). At the beginning of the annealing process, when $t = 0$, we have $A(0) > B(0)$, which puts the system in a quantum ground state where each spin is delocalized and is a superposition of its classical counterparts. During annealing, $A(s)$ gradually decreases, while $B(s)$ increases until T , when the final state of the qubits corresponds to a low-energy solution. In the final annealing phase, the Hamiltonian consists solely of the $B(s)$. Eventually, it takes the form of the classical Hamiltonian, in which every possible classical configuration of bits corresponds to an eigenstate, and the eigenenergy is related to the objective function that was introduced into the system. Both SA and QA will be executed using D-Wave Samplers. Additionally, in QA, we will use automatic embedding

of our problem into the target quantum machine.

Our approach based on binarized values of neuron activation probability in RBM's hidden layer leads to a maximum number of unique returned labels equal to 2^N , where $N \in \{3, 4, \dots, 28\}$. Since there are seven classes in the HyperBlood dataset after our preprocessing, we see that the RBM will return more labels than in the dataset. Accordingly, to [4], RBM may be used to build relational trees and then use these hierarchies to divide the data into groups and subgroups. We will use the structure clustered by RBM to create a distance matrix, which will serve as an input to the Agglomerative Hierarchical Clustering (AHC) algorithm. By specifying the target number of clusters for the AHC, we aim to obtain clusterization that will be useful in our segmentation task.

Having the RBM models trained by CD-1, SA, and QA and identified which are the best in the context of the observable metrics, as the last step, we proceed to create segmentation images using these three RBM models and AHC.

Chapter 6

Experiments and results

This chapter reports on the experiments and results obtained for the segmentation of multispectral images using the proposed neural network models. Model training was carried out using different approaches: the traditional contrastive divergence (CD-1) algorithm and an approach based on simulated annealing (SA) and quantum annealing (QA) to improve optimization efficiency. The following subsections cover an analysis of the comparative performance of the both methods in the context of multispectral image segmentation.

6.1 Dataset

In this work, we will focus on the HyperBlood dataset [50]. The dataset consists of fourteen multispectral images showing a mock-up of a scene with blood and visually similar to human eye substances. The images, collected over three weeks, vary in background composition and lighting conditions. Each image consists of 120 spectral bands, and each image is annotated with class labels indicating the presence of blood and other substances. This dataset was designed to support the development and evaluation of machine learning algorithms for multispectral blood detection and classification [50]. The dataset consists of eight classes: background, blood, ketchup, artificial blood, beetroot juice, poster paint, tomato concentrate, and acrylic paint. Figure 6.1 shows an overview of the classes present in the HyperBlood dataset. Figure ?? shows an example of selected bands from two multispectral images from the studied dataset.

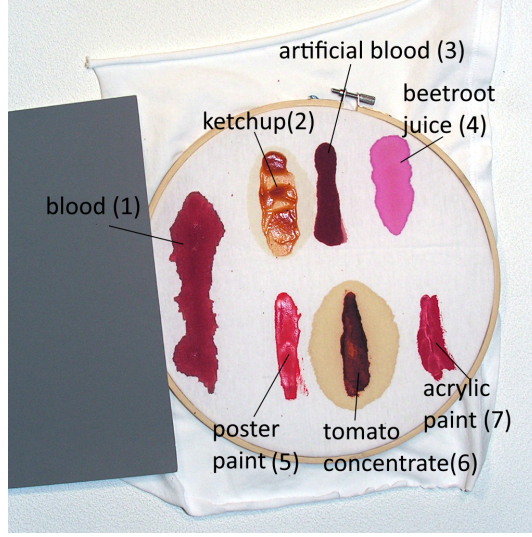


Figure 6.1: An image from the dataset documentation showing an overview of the classes present in the dataset [50].

6.2 Classical Model Training and Evaluation

Throughout this section, we will present training hyperparameters and resulting loss curves for each trained model containing LBAE and RBM models, which were trained with the CD-1 algorithm, simulated annealing, and quantum annealing. We will also present the evaluation metrics that we have obtained that justify proceeding further with the model, which we consider superior to other models. Obtained segmentation images and metrics computed for those images will be presented in the next section of this work.

At first, we will begin with training Latent Bernoulli Autoencoder (LBAE). As mentioned in Chapter 5, our approach relies on the work done in [51]. Since we changed only convolutional layers from two to one-dimensional, we will keep fixed hyperparameters of these layers as they were in [51]. Our experiments focused on studying the influence of changing batch size and learning rate.

The following values of batch size $b_i \in \{4, 8, 16\}$ and learning rate $\eta_i \in \{10^{-2}, 10^{-3}, 10^{-4}\}$ were chosen as a standard values [28]. For each combination of these parameters, LBAE training was conducted, ultimately yielding nine trained models. Each model's performance was evaluated on a test dataset using Euclidean distance and spectral angle distance (SAD) metrics. Figure ?? shows values of both mentioned metrics for each trained LBAE model. Figure ?? shows the learning curve of the model that achieved the lowest value of both Euclidean distance and spectral angle distance metrics.

We will proceed with the LBAE model that obtained the lowest values of both metrics, that is the model trained with hyperparameters $b_i = 4$ and $\eta_i = 10^{-3}$.

Having trained the LBAE model, we determine a baseline for Restricted Boltzmann

Machines (RBM) clusterization that we aim to surpass. For this purpose, we will use the k -means algorithm to perform clusterization on the test dataset for both scenarios, first for the data processed as described in Chapter 5 and the data encoded by the LBAE encoder. To avoid the impact of how the centroids are initialized, we conducted the clustering ten times using different random seeds. The metrics introduced in the Section ?? were computed for the resulting clustering. We will consider these metrics as baselines, visualized in the metrics plots computed for clustering using RBM.

We then proceeded with the RBM part and started with the CD-1 training procedure. Similar to the case of LBAE, we keep hyperparameters as they were in [51], except for one of them — a number of neurons in the hidden layer. We directed our efforts to explore the impact of the number of neurons in the RBM’s hidden layer. RBM training experiments were conducted for the following number of neurons in hidden layer $h_i \in \{3, 4, \dots, 28\}$, and those experiments were repeated ten times for different weights initialization. After the training process, we obtained clusterization metrics, and their results are illustrated in Figures ??–??.

Due to limited resources, we would like to proceed with further experiments involving simulated and quantum annealing using the best RBM model from those we obtained. However, it is unclear which of those models is better than others. To face the problem of choosing one model, we will use a V-measure, a meta-metric, whose value depends on homogeneity, completeness, and an additional β parameter. Authors of [53] do not suggest how to choose β value, however it is said that $\beta \in [0, 1]$ promotes homogeneity. We will conduct an experiment, exploring how frequently, for which model we obtain the greatest value of V-measure for $\beta_i \in \{0, 0.01, \dots, 1\}$. This experiment concluded that the RBM model with 23 neurons in the hidden layer was the most promising for the segmentation task. Figure 6.2 shows the learning curve for that model.

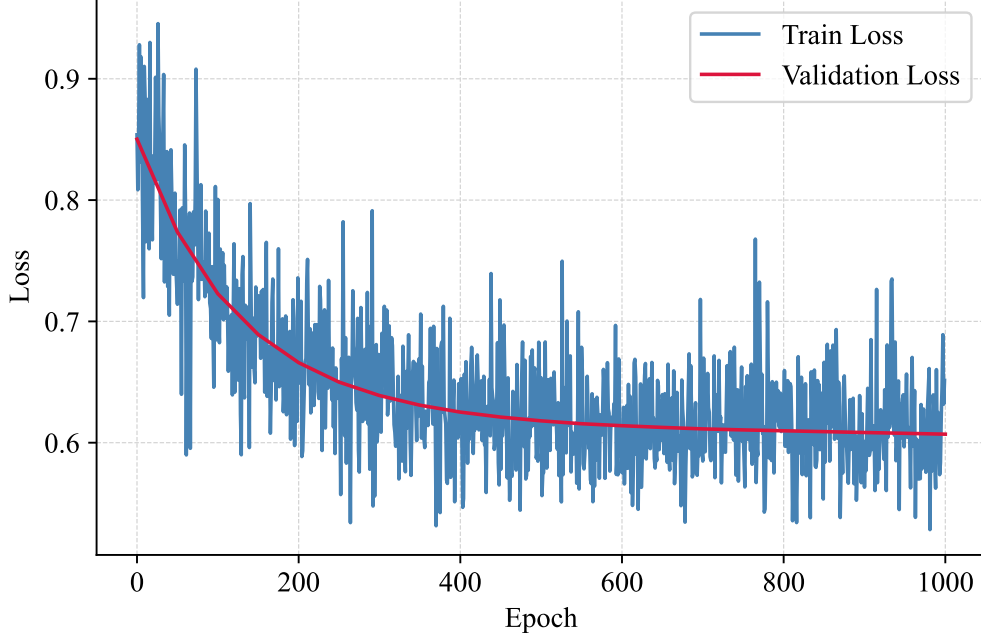


Figure 6.2: Learning curve of the RBM from the contrastive divergence (CD-1) training. The blue line represents the loss value on the training dataset, and the red line represents the loss value on the validation dataset.

After the RBM training, we faced the problem of RBM returning more unique labels than the target number of labels on ground-truth images. Following the idea that the RBM returned structure is hierarchical [4], we could pass this structure to another algorithm, known as Agglomerative Hierarchical Clustering (AHC), and specify the target number of clusters.

For the segmentation images that we will create, we want to refer to a reliable baseline; for this task, we will again use the k -means algorithm. Figure 6.3a illustrates the clustering using k -means, and Figure 6.3b illustrates the ground truth image. Table 6.1 shows metrics comparison for created segmentation images.

After obtaining baseline segmentation, we proceeded to create segmentation images using our network, which consists of an LBAE encoder, RBM, and AHC. These images are presented in Figure 6.4a and Figure 6.4b.

6.3 Quantum Model Training and Evaluation

With the RBM hyperparameters determined, the next step of the project was to test the implemented training algorithm using a simulation annealer. For this task, we used D-Wave’s implementation of a simulated annealing sampler [19]. The training was repeated

ten times for initialization with different weights, this time only for an RBM model containing 23 neurons in the hidden layer. We decided to save a model after every other hundred training epochs to execute an insightful analysis of model performance in the context of clusterization metrics. Obtained clusterization metrics and their results are shown in Figures ??–??.

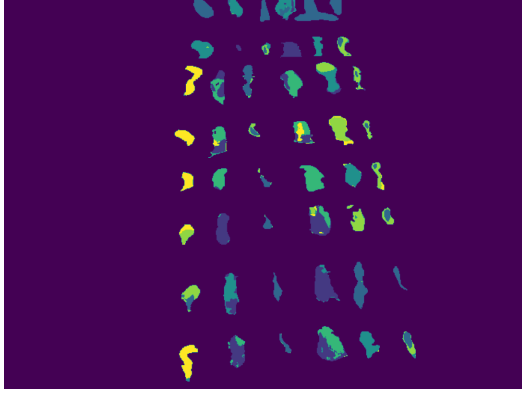
Wanting to select the best model, we performed the procedure described in ?? using the V-measure metric. This procedure concluded that the best one, from the obtained models, is the model saved on epoch 200. The learning curve of the RBM trained using the simulated annealer is shown in Figure ??.

At this point, both RBM models have been trained with CD-1 and simulated annealing, so we proceed to train our model using quantum annealing. Again, we use the sampler provided by D-Wave [19] and the automatic embedding of our problem into the target QPU SolverAdvantage 5.4. The training was repeated ten times for initialization with different weights, again only for an RBM model containing 23 neurons in the hidden layer. We decided to save a model after every other hundred training epochs. Obtained clusterization metrics and their results are shown in Figures ??–??.

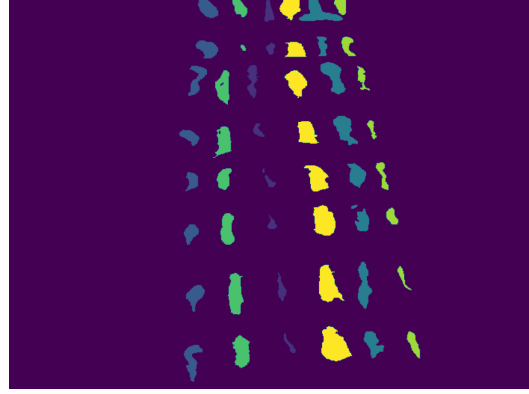
As before, we repeat the procedure using the V-measure to obtain the model with the best performance in the context of evaluation metrics. This procedure concluded that the best one, from the obtained models, is the model saved on epoch 300. The learning curve of the RBM trained using the quantum annealer is shown in Figure ??.

6.4 Segmentation Results

In this section we will present all obtained segmentations, for the image illustrated in Figure ?? and evaluation metrics computed on those segmentations. We will start showing our experiments results with our baseline segmentation obtained by using k -means algorithm on the Figure 6.3a and ground truth image on the Figure 6.3b. Next, we will present segmentations obtained by using our proposed model trained with contrastive divergence (CD-1) and followed by AHC using both linkage methods — complete and average, those will be respectively Figure 6.4a and Figure 6.4b. Similarly we will present segmentations for models trained with simulated annealing (SA) and quantum annealing (QA). For simulated annealing obtained images are Figure 6.4c and Figure 6.4d, and for quantum annealing obtained images are Figure 6.4e and Figure 6.4f.



(a) k -means segmentation.



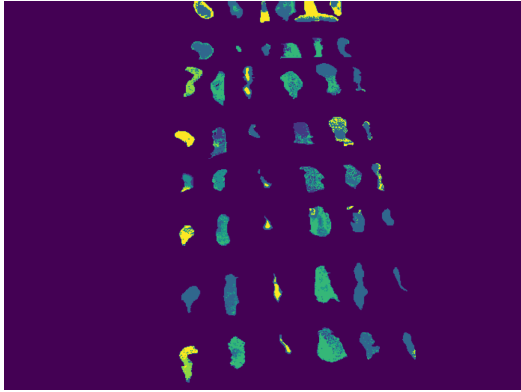
(b) Ground truth.

Figure 6.3: Baseline segmentation by k -means and ground truth.

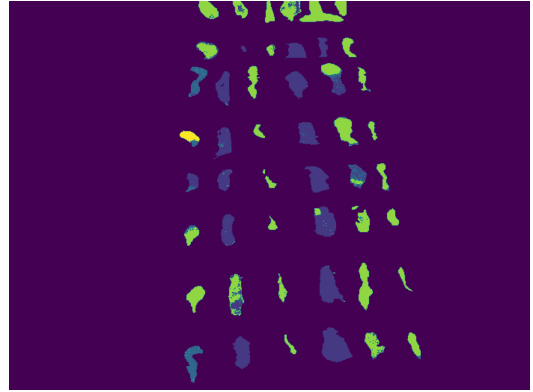
A summary of the computed evaluation metrics such as homogeneity, completeness, adjusted rand score (ARS) and rand score (RS) for each segmentation obtained is presented collectively in Table 6.1.

6.5 Conclusions

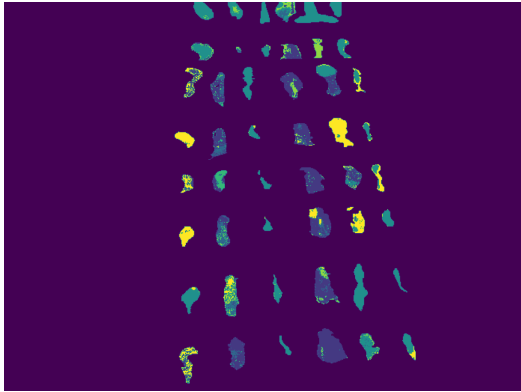
Analyzing obtained metrics on the segmentation images, we notice that almost all metrics improve their value for RBM trained with QA, and if they are not improved, they are close to the best-obtained value. For a model that does not use AHC, we note the consistent improvement of the metrics while changing the RBM training algorithm from CD-1 through SA to QA. For the metrics homogeneity and ARS, the RBM model trained with QA surpassed the baseline value, which was set by the k -means algorithm. The use of the AHC algorithm in combination with RBM affects clustering characteristics. First, AHC improves the completeness metric, especially when using the RBM version trained with Quantum Annealing. At the same time, adding AHC reduces the clusters' homogeneity.



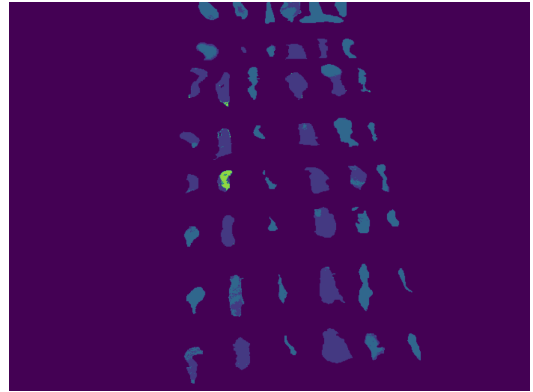
(a) Segmentation by RBM trained by CD-1 with AHC using complete linkage.



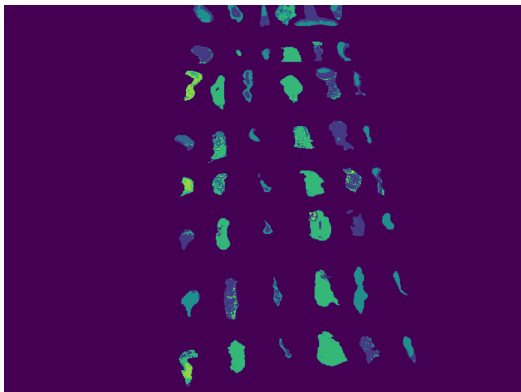
(b) Segmentation by RBM trained by CD-1 with AHC using average linkage.



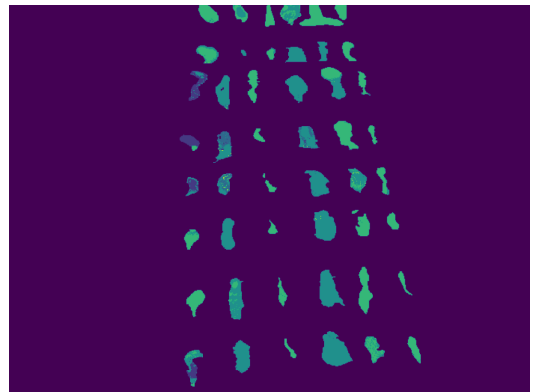
(c) Segmentation by RBM trained by SA with AHC using complete linkage.



(d) Segmentation by RBM trained by SA with AHC using complete linkage.



(e) Segmentation by RBM trained by QA with AHC using complete linkage.



(f) Segmentation by RBM trained by QA with AHC using complete linkage.

Figure 6.4: Comparison of the obtained segmentations.

Table 6.1: Comparison of clustering evaluation metrics for baseline segmentation and segmentation produced by our models. Here, RBM+AHC-C denotes the RBM with AHC using complete linkage, and RBM+AHC-A denotes the RBM with AHC using average linkage.

Metric	Training	k -means	RBM	RBM+AHC-C	RBM+AHC-A
Homogeneity	–	0.368	–	–	–
	CD-1	–	0.492	0.232	0.243
	SA	–	0.510	0.231	0.171
	QA	–	0.505	0.254	0.260
Completeness	–	0.354	–	–	–
	CD-1	–	0.239	0.257	0.416
	SA	–	0.244	0.266	0.381
	QA	–	0.282	0.319	0.445
ARS	–	0.244	–	–	–
	CD-1	–	0.163	0.129	0.242
	SA	–	0.180	0.199	0.181
	QA	–	0.308	0.284	0.246
Rand Score	–	0.769	–	–	–
	CD-1	–	0.793	0.690	0.660
	SA	–	0.799	0.711	0.590
	QA	–	0.790	0.729	0.656

Chapter 7

Summary

This work proposes a hybrid neural network architecture — Latent Bernoulli Autoencoder’s encoder connected with Restricted Boltzmann Machines for multispectral image segmentation. Firstly, the LBAE’s encoder part is utilized for dimensionality reduction and encoding spectral data to a binary representation, which is necessary for RBM. Secondly, we use RBM to perform clustering on the encoded spectral pixels.

The proposed architecture and experiments confirmed the effectiveness of applying quantum annealing techniques and RBM models for the segmentation of multispectral images since we achieved better segmentations, in terms of observed metrics than the baseline segmentation obtained using standard algorithm — k -means. The results suggest the potential for further research on using quantum annealing in the image processing field.

Bibliography

- [1] I. J. Goodfellow, J. Pouget-Abadie, M. Mirza, *et al.*, *Generative adversarial networks*, 2014.
- [2] A. Vaswani, N. Shazeer, N. Parmar, *et al.*, *Attention is all you need*, 2023.
- [3] P. Smolensky, “Information processing in dynamical systems: Foundations of harmony theory”, in *Parallel Distributed Processing: Explorations in the Microstructure of Cognition*, D. E. Rumelhart and J. L. McClelland, Eds., vol. 1, MIT Press, 1986, pp. 194–281.
- [4] A. Decelle, B. Seoane, and L. Rosset, “Unsupervised hierarchical clustering using the learning dynamics of restricted Boltzmann machines”, *Physical Review E*, vol. 108, 2023.
- [5] V. Dixit, R. Selvarajan, M. A. Alam, T. S. Humble, and S. Kais, “Training restricted Boltzmann machines with a D-Wave quantum annealer”, *Frontiers in Physics*, vol. 9, 2021.
- [6] Y. Wang and X. Wang, “Image segmentation application of conditional convolution restricted Boltzmann machine”, in *2019 2nd International Conference on Intelligent Autonomous Systems (ICoIAS)*, 2019.
- [7] M. Agn, O. Puonti, P. Munck af Rosenschold, I. Law, and K. Van Leemput, “Brain tumor segmentation using a generative model with an RBM prior on tumor shape”, 2016.
- [8] P. Wang, S. Wang, Y. Zhang, and X. Duan, “Multispectral image under tissue classification algorithm in screening of cervical cancer”, *Journal of Healthcare Engineering*, 2022.
- [9] J. Fajtl, V. Argyriou, D. Monekosso, and P. Remagnino, “Latent Bernoulli autoencoder”, in *Proceedings of the 37th International Conference on Machine Learning*, PMLR, 2020.

- [10] D. Mengu, A. Tabassum, M. Jarrahi, and A. Ozcan, “Snapshot multispectral imaging using a diffractive optical network”, *Light: Science & Applications*, vol. 12, no. 1, 2023.
- [11] S. Sifnaios, G. Arvanitakis, F. K. Konstantinidis, G. Tsimiklis, A. Amditis, and P. Frangos, *A deep learning approach for pixel-level material classification via hyperspectral imaging*, 2024.
- [12] S. Morchhale, V. P. Pauca, R. J. Plemmons, and T. C. Torgersen, “Classification of pixel-level fused hyperspectral and lidar data using deep convolutional neural networks”, in *2016 8th Workshop on Hyperspectral Image and Signal Processing: Evolution in Remote Sensing (WHISPERS)*, 2016.
- [13] J. González-Santiago, F. Schenkel, W. Gross, and W. Middelmann, “Deep self-supervised pixel-level learning for hyperspectral classification”, in *2022 12th Workshop on Hyperspectral Imaging and Signal Processing: Evolution in Remote Sensing (WHISPERS)*, 2022.
- [14] C. Yan, X. Fan, J. Fan, *et al.*, “HyFormer: Hybrid transformer and CNN for pixel-level multispectral image land cover classification”, *International Journal of Environmental Research and Public Health*, vol. 20, 2023.
- [15] X. Chen, “Hyperspectral image classification using pixel-level convolutional neural network based on attention mechanism”, in *Proceedings of the 5th International Conference on Computer Information and Big Data Applications*, ser. CIBDA '24, Association for Computing Machinery, 2024, pp. 819824.
- [16] R. Pereira da Silva, “Gate-based quantum computing: An overview”, *SSRN Electronic Journal*, Feb. 2023.
- [17] T. Kadowaki and H. Nishimori, “Quantum annealing in the transverse Ising model”, *Physical review. E, Statistical physics, plasmas, fluids, and related interdisciplinary topics*, vol. 58, 1998.
- [18] M. Schuld and F. Petruccione, *Machine Learning with Quantum Computers*, 2nd. Springer Nature Switzerland AG, 2021.
- [19] *D-Wave documentation*, Accessed: 6th October 2024, D-Wave Systems Inc., 2024. [Online]. Available: <https://docs.dwavesys.com/>.
- [20] J. Ma, Y. He, F. Li, L. Han, C. You, and B. Wang, “Segment anything in medical images”, *Nature Communications*, vol. 15, no. 1, 2024.
- [21] R. Kemker, U. B. Gewali, and C. Kanan, “Earthmapper: A tool box for the semantic segmentation of remote sensing imagery”, *IEEE*, 2018.

- [22] S. Cakir, M. GauSS, K. Happeler, Y. Ounajjar, F. Heinle, and R. Marchthaler, “Semantic segmentation for autonomous driving: Model evaluation, dataset generation, perspective comparison, and real-time capability”, *arXiv preprint*, 2022.
- [23] N. R. Pal and S. K. Pal, “A review on image segmentation techniques”, *Pattern Recognition*, vol. 26, no. 9, 1993.
- [24] J. Chen, H. Shao, and C. Hu, “Image segmentation based on mathematical morphological operator”, in *2017 International Conference on Advanced Mechatronic Systems (ICAMechS)*, 2017.
- [25] V. Badrinarayanan, A. Kendall, and R. Cipolla, “Segnet: A deep convolutional encoder-decoder architecture for image segmentation”, *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 39, no. 12, 2017.
- [26] N. Dhanachandra, K. Manglem, and Y. J. Chanu, “Image segmentation using k-means clustering algorithm and subtractive clustering algorithm”, *Procedia Computer Science*, vol. 54, 2015.
- [27] P. Mohanta, D. Mukherjee, and S. Acton, “Agglomerative clustering for image segmentation”, in *2002 International Conference on Pattern Recognition*, vol. 1, 2002.
- [28] S. Raschka and V. Mirjalili, *Python Machine Learning: Machine Learning and Deep Learning with Python, Scikit-learn, and TensorFlow 2*, 3rd. Packt Publishing, 2019.
- [29] I. Goodfellow, Y. Bengio, and A. Courville, *Deep learning*. MIT Press, 2016.
- [30] W. S. McCulloch and W. Pitts, “A logical calculus of the ideas immanent in nervous activity”, *Bulletin of Mathematical Biophysics*, vol. 5, no. 4, 1943.
- [31] F. Rosenblatt, “The perceptron: A probabilistic model for information storage and organization in the brain”, *Psychological Review*, vol. 65, no. 6, 1958.
- [32] K. Bindra and A. Mishra, “A detailed study of clustering algorithms”, in *2017 6th International Conference on Reliability, Infocom Technologies and Optimization (Trends and Future Directions) (ICRITO)*, 2017.
- [33] W. Wang, Y. Huang, Y. Wang, and L. Wang, *Generalized autoencoder: A neural network framework for dimensionality reduction*, 2014.
- [34] T. Tieleman, “Training restricted Boltzmann machines using approximations to the likelihood gradient”, in *Proceedings of the 25th International Conference on Machine Learning*, Association for Computing Machinery, 2008.
- [35] N. Bratchell, “Cluster analysis”, *Chemometrics and Intelligent Laboratory Systems*, vol. 6, no. 2, 1989.

- [36] F. F. Flöther, “The state of quantum computing applications in health and medicine”, *Research Directions: Quantum Technologies*, vol. 1, 2023.
- [37] U. Ahmed, T. Sipola, and J. Hautamäki, “Cyber protection applications of quantum computing: A review”, *European Conference on Cyber Warfare and Security*, vol. 23, no. 1, 2024.
- [38] R. Orús, S. Muga, and E. Lizaso, “Quantum computing for finance: Overview and prospects”, *Reviews in Physics*, vol. 4, 2019.
- [39] F. Phillipson, *Quantum computing in logistics and supply chain management an overview*, 2024.
- [40] A. Datta, S. T. Flammia, and C. M. Caves, “Entanglement and the power of one qubit”, *Physical Review A*, 2005.
- [41] D. McMahon, *Quantum Computing Explained*. John Wiley & Sons, Inc., 2008.
- [42] K. Jaowiecki, “Validation and benchmarking of quantum annealing technology”, Ph.D. dissertation, IITIS PAN, 2024.
- [43] E. Farhi, J. Goldstone, S. Gutmann, and M. Sipser, *Quantum computation by adiabatic evolution*, 2000.
- [44] V. Kumar, G. Bass, C. Tomlin, and J. Dulny, *Quantum annealing for combinatorial clustering*, 2018.
- [45] P. Gomb, P. Sadowski, O. Faucoz, *et al.*, “Hyper-spectral image classification using adiabatic quantum computation”, 2023.
- [46] C. S. Calude, E. Calude, and M. J. Dinneen, “Guest column: Adiabatic quantum computing challenges”, vol. 46, 2015.
- [47] D. Ferrer-Balas, M. L. Maspoch, and O. O. Santana Perez, “Influence of annealing on the microstructural, tensile and fracture properties of polypropylene films”, *Polymer*, 2001.
- [48] R. Benedetti, M. M. Dickson, G. Espa, F. Pantalone, and F. Piersimoni, *A simulated annealing-based algorithm for selecting balanced samples*, 2021.
- [49] W. Vinci and D. A. Lidar, “Non-stoquastic hamiltonians in quantum annealing via geometric phases”, *npj Quantum Information*, vol. 3, no. 1, 2017.
- [50] M. Romaszewski, P. Gomb, A. Sochan, and M. Cholewa, “A dataset for evaluating blood detection in hyperspectral images”, *Forensic Science International*, vol. 320, 2021.

- [51] . Pawela and K. Jaowiecki, *QBM4EO — Supervised quantum machine learning system for earth land cover understanding*. 2022.
- [52] *Pytorch documentation*, Accessed: 6th October 2024, 2024. [Online]. Available: <https://pytorch.org/docs/stable/index.html>.
- [53] A. Rosenberg and J. Hirschberg, “V-measure: A conditional entropy-based external cluster evaluation measure”, in *Proceedings of the 2007 Joint Conference on Empirical Methods in Natural Language Processing and Computational Natural Language Learning (EMNLP-CoNLL)*, Association for Computational Linguistics, 2007.